OpenCV CUDA Object Detection

Setup, Issues & Resolutions on Jetson AGX Orin

Platform: NVIDIA Jetson AGX Orin | CUDA 11.4 | OpenCV 4.13.0 | Python 3.8

1. Project Overview

The goal of this project was to run a YOLOv8-based object detection model (trained to detect apples) using pure OpenCV with CUDA acceleration on a NVIDIA Jetson AGX Orin. The key requirement was to eliminate PyTorch and torchvision from the inference pipeline entirely, using only OpenCV's DNN module with CUDA backend.

The pipeline integrates with an Intel RealSense depth camera to perform 3D localization of detected objects and send coordinates to a remote server.

2. System Setup

2.1 Hardware

Component	Details
Device	NVIDIA Jetson AGX Orin
GPU	Ampere architecture (integrated)
RAM	30 GB
Camera	Intel RealSense D400 series
Architecture	ARM v8 (aarch64)

2.2 Software Stack

Component	Version / Details
OS	Ubuntu (Jetson)
Python	3.8.10
CUDA Toolkit	11.4
OpenCV	4.13.0-dev (built from source)
PyTorch	2.4.1 (used only for ONNX export, not inference)
pyrealsense2	Intel RealSense SDK

3. Installation & Configuration

3.1 Discovering the System

nvidia-smi was not found initially. The GPU was identified via:

```
lspci | grep -i nvidia
```

This returned device 229e — an NVIDIA PCIe bridge indicating a Jetson platform. Confirmed with:

```
cat /etc/nv_tegra_release
```

GPU monitoring on Jetson uses tegrastats instead of nvidia-smi:

```
tegrastats
```

3.2 Fixing CUDA PATH

Both nvidia-smi and nvcc were not found due to missing PATH entries. Three CUDA folders were found in /usr/local: cuda, cuda-11, and cuda-11.4. The following was added to ~/.bashrc:

```
export CUDA_HOME=/usr/local/cuda-11.4
export PATH=$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
source ~/.bashrc
```

After this, nvcc --version correctly returned release 11.4.

3.3 OpenCV with CUDA

Standard pip wheels (opencv-python, opencv-contrib-python) do not include CUDA support. OpenCV was already built from source on this Jetson and located at:

```
/usr/local/lib/python3.8/site-packages/cv2/
```

However Python could not import it because it was looking in dist-packages instead of site-packages. Fixed with a symlink:

```
sudo ln -s /usr/local/lib/python3.8/site-packages/cv2
/usr/local/lib/python3.8/dist-packages/cv2
```

Verified CUDA was enabled:

```
python3 -c "import cv2; print(cv2.cuda.getCudaEnabledDeviceCount())"  # returns 1
```

3.4 Enabling CUDA Backend in OpenCV DNN

To use CUDA for inference, set the backend and target after loading the model:

```
net.setPreferableBackend(cv2.dnn.DNN_BACKEND_CUDA)
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CUDA)
```

For FP16 inference (faster, slight accuracy tradeoff):

```
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CUDA_FP16)
```

4. Model Conversion: .pt to ONNX

4.1 Why Convert?

OpenCV's DNN module cannot load PyTorch .pt files directly. The model must be exported to ONNX format. This is a one-time step — after conversion, ultralytics is no longer needed at inference time.

4.2 Export Command

```
from ultralytics import YOLO
YOLO('my_model.pt').export(format='onnx', imgsz=640, opset=17, simplify=False)
```

The simplify=False flag was required because onnxslim (used by simplify=True) crashed with a core dump on the Jetson ARM platform.

4.3 Verify ONNX Loads in OpenCV

```
net = cv2.dnn.readNetFromONNX('my_model.onnx')
print(net.getUnconnectedOutLayersNames())
```

5. YOLOv8 Output Format

5.1 Output Shape

YOLOv8 ONNX exports with output shape (1, 6, 8400) for a 2-class model. The structure is:

Dimension	Meaning
1	Batch size
6	4 bbox coords (cx, cy, w, h) + 2 class scores
8400	Number of anchor predictions

5.2 Parsing the Output

The output must be transposed before iterating over detections:

```
output = np.squeeze(outputs[0]).T   # shape becomes (8400, 6)
scores = det[4:]                    # class scores start at index 4 (not 5 like YOLOv4)
```

Note: YOLOv4 used det[5:] because index 4 was an objectness score. YOLOv8 removed this — class scores start at index 4 directly.

5.3 Coordinate Format

YOLOv8 ONNX outputs coordinates already in pixel space (not normalized 0-1). Do NOT multiply by image width/height:

```
# WRONG:  cx = det[0] * INPUT_SIZE[0]
# CORRECT: cx = det[0]
```

6. Critical Fix: Letterbox Preprocessing

6.1 The Problem

After switching from ultralytics to OpenCV DNN, the model stopped detecting objects. Ultralytics preprocesses images using letterboxing (resize with padding to maintain aspect ratio). OpenCV's blobFromImage simply squishes the image to the target size, distorting it enough that the model fails to detect.

6.2 The Fix

A letterbox function was implemented to match ultralytics preprocessing exactly:

```
def letterbox(image, target_size=(640, 640)):
    h, w = image.shape[:2]
    scale = min(target_size[0]/w, target_size[1]/h)
    new_w, new_h = int(w*scale), int(h*scale)
    resized = cv2.resize(image, (new_w, new_h))
    canvas = np.full((640, 640, 3), 114, dtype=np.uint8)  # 114 = ultralytics pad
color
    pad_x = (640 - new_w) // 2
    pad_y = (640 - new_h) // 2
    canvas[pad_y:pad_y+new_h, pad_x:pad_x+new_w] = resized
    return canvas, scale, pad_x, pad_y
```

6.3 Coordinate Correction

After letterboxing, predicted coordinates are in letterboxed space and must be mapped back to the original frame:

```
cx = (det[0] - pad_x) / scale
cy = (det[1] - pad_y) / scale
bw = det[2] / scale
bh = det[3] / scale
```

7. Issues & Resolutions

Issue	Cause	Fix
pip install opencv-contrib-python returns no CUDA	Standard pip wheels are CPU-only	Use OpenCV built from source or cudawarped wheels
nvidia-smi: command not found	CUDA not in system PATH	Add CUDA_HOME/bin to PATH in ~/.bashrc
cv2 module not found	Python looking in dist-packages, cv2 in site-packages	Symlink site-packages/cv2 to dist-packages/cv2
CUDA devices: 0	LD_LIBRARY_PATH missing CUDA libs	Add cuda-11.4/lib64 to LD_LIBRARY_PATH
onnxslim core dump on export	onnxslim crashes on Jetson ARM	Use simplify=False in YOLO export
IndexError: index out of bounds	Old YOLOv4 parsing used det[5:], YOLOv8 uses det[4:]	Use det[4:] for class scores + transpose output
cv2.error: empty ROI in cvtColor	Bounding box extends outside frame boundaries	Clamp x1/y1/x2/y2 to frame dimensions before crop
Model detects with ultralytics but not OpenCV	blobFromImage squishes image, breaking aspect ratio	Implement letterbox() with pad color 114 to match ultralytics
Bounding box drawn off screen	Coords multiplied by 640 but already in pixels	Remove INPUT_SIZE multiplication from coord conversion
No box drawn despite model detecting	THRESHOLD=160 filtered out apple ROI brightness	Lower THRESHOLD to 50
ImportError: cannot allocate memory in static TLS	ARM Jetson torch libgomp conflict	export LD_PRELOAD=.../libgomp-804f19d4.so.1.0.0

8. Final Pipeline

8.1 Flow

- RealSense captures aligned color + depth frames
- Color frame is letterboxed to 640x640 with grey padding (value=114)
- OpenCV DNN runs YOLOv8 inference on CUDA backend
- Output (1,6,8400) is transposed to (8400,6) and parsed
- NMS filters overlapping boxes
- Coordinates mapped back from letterboxed to original frame space
- Bounding box clamped to frame boundaries
- ROI brightness and depth validity checks applied

- Depth sensor deprojects pixel to 3D XYZ with camera offsets
- After DETECTION_FRAMES stable detections, coordinates averaged and sent via socket

8.2 Key Parameters

Parameter	Value / Description
CONF_THRESH	0.5 — minimum detection confidence
NMS_THRESH	0.4 — non-maximum suppression threshold
THRESHOLD	50 — minimum ROI brightness to accept detection
DETECTION_FRAMES	4 — frames required before sending coordinates
delta	0.15m — maximum std dev for stable coordinate averaging
Z_OFFSET	0.13m — depth offset correction
INPUT_SIZE	(640, 640) — model input resolution

9. Quick Reference Commands

Monitor GPU

```
tegrastats
```

Verify CUDA in OpenCV

```
python3 -c "import cv2; print(cv2.cuda.getCudaEnabledDeviceCount())"
```

Export Model to ONNX

```
python3 -c "from ultralytics import YOLO; YOLO('my_model.pt').export(format='onnx',
imgsz=640, opset=17, simplify=False)"
```

Check Model Output Shape

```
python3 -c "import cv2, numpy as np; net=cv2.dnn.readNetFromONNX('my_model.onnx');
net.setInput(cv2.dnn.blobFromImage(np.zeros((480,640,3),dtype=np.uint8),1/255,(640,
640))); [print(o.shape) for o in net.forward(net.getUnconnectedOutLayersNames())]"
```

Run Detection

```
python3 cam_test.py
```